Passive Alcohol Detection System

GitHub-ready public project summary

This PDF is designed for the docs/ folder of your GitHub repository. It presents the project overview, general working, architecture, demo code, sample outputs, applications and technology stack without exposing patent claims or complete technical specification.

Repository use	Upload as docs/project_summary.pdf and link it from README.md.
Status	Ongoing / simulation and prototype development. Replace sample screenshots with your real outputs when available.
IP notice	Patent draft, claims, exact novelty points and complete specification are intentionally excluded.

1. Project Overview

The Passive Alcohol Detection System is an embedded safety-monitoring concept that detects alcohol vapor in the surrounding environment and combines it with human presence information to generate a local alert. The system is intended for non-intrusive monitoring in controlled areas such as workplaces, transport checkpoints, laboratories, campuses and industrial gates.

Unlike a breath analyzer, this concept does not require a person to blow into a device. It uses passive sensing, edge processing and alert outputs to support early safety warnings. This GitHub version is a public-safe engineering summary and does not include the complete patent specification or claim language.

Key Goals

- Detect possible alcohol vapor presence using a sensing module.
- Check whether human presence or motion is detected near the device.
- Use simple embedded decision logic to classify the situation as normal, warning or alert.
- Activate local indicators such as LED and buzzer when threshold conditions are met.
- Log or transmit status values to an optional IoT dashboard for monitoring.

Public-safe scope: this document uses high-level descriptions, simplified diagrams and demo code. Replace sample values with validated readings after practical testing.

Problem Statement

Manual alcohol checking is not scalable in crowded or high-throughput environments. Camera-based systems may raise privacy concerns, and handheld breath analyzers require direct cooperation. A compact embedded sensing system can support safety teams by providing early, non-contact indication when vapor level and human presence conditions occur together.

2. Technologies Used

Category	Tools / Components	Purpose
Embedded Controller	ESP32 / Arduino-compatible controller	Sensor reading, threshold logic, alert control and data output.
Alcohol Vapor Input	Generic alcohol vapor sensor for demo/simulation	Measures analog vapor level for prototype demonstration.
Human Presence	PIR / ultrasonic / motion sensing module	Checks whether a person is near the sensing area.
Environment	Temperature and humidity sensor	Supports compensation and context awareness.
Alert Hardware	LED, buzzer, optional relay	Provides local indication when risk condition is detected.
Software	Arduino IDE, Embedded C/C++, Proteus or MATLAB/Simulink	Code development, simulation and validation.
IoT / Logging	Serial monitor, JSON stream, cloud dashboard optional	Stores or visualizes readings and event status.

Recommended GitHub Topics

`iot` `embedded-systems` `esp32` `arduino` `sensor-fusion` `safety-system` `alcohol-detection` `proteus` `matlab` `edge-computing`

3. Basic Block Diagram

The diagram below is a simplified public-safe architecture that can be used in GitHub documentation. It avoids detailed patent claims and focuses only on the high-level prototype workflow.

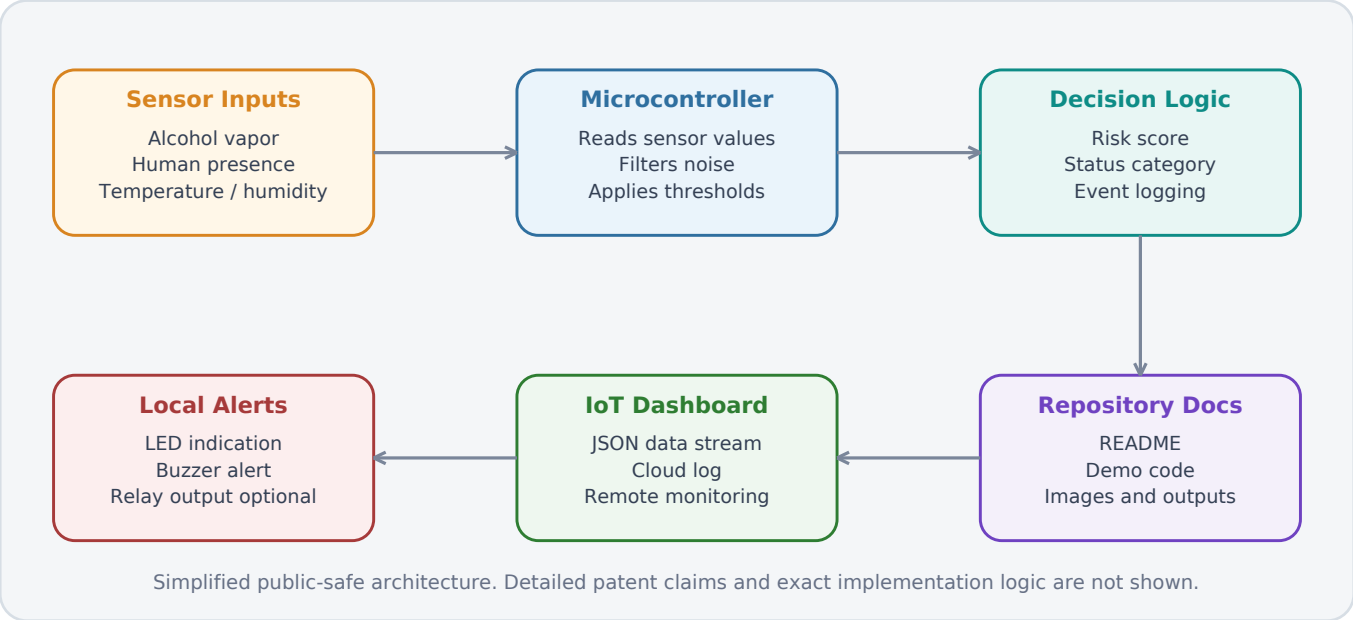

Figure 1. Basic public-safe architecture for repository documentation.

Module Description

Module	Description
Sensor Inputs	Collect vapor level, human presence and environmental readings.
Microcontroller	Reads sensor values, filters noise and applies threshold-based logic.
Decision Logic	Combines alcohol level and presence status to generate normal, warning or alert states.
Local Alerts	Activates LED or buzzer for immediate safety indication.
IoT Dashboard	Optional logging or visualization through serial output or cloud dashboard.

4. General Working

The system works in a continuous monitoring loop. It first initializes the controller and sensors, then reads alcohol vapor, presence and environmental inputs. The controller compares the readings with defined thresholds and generates a status output.

Figure 2. Simplified working flow.

Step-by-step operation

1. Initialization: The controller configures input and output pins, starts serial communication and sets initial system status.

2. Sensor acquisition: The alcohol vapor sensor, human presence sensor and optional environmental sensor readings are collected.

3. Preprocessing: Raw analog values are converted into usable ranges. Noise can be reduced using averaging or filtering.

4. Decision logic: Alcohol level and human presence are checked together. Alert is generated only when risk conditions are met.

5. Alert output: LED and buzzer indicate the current status locally.

6. Logging: Readings and status are printed in JSON-like format for dashboards or future cloud integration.

Important: threshold values in the demo code are placeholders. Use calibration values obtained from your own sensor testing before showing final performance claims.

5. Simulation Screenshots and Output Images

No real simulation screenshots were attached with this request, so this PDF includes GitHub-ready screenshot sections and sample output previews. Replace the sample panels below with your actual Proteus, MATLAB/Simulink, serial monitor or dashboard screenshots before final submission.

Simulation Screenshot Slot

```
Add: Proteus / MATLAB / Simulink circuit screenshot
Show: controller + sensor inputs + LED/buzzer output
Caption: Simulation setup for alcohol detection demo

Tip: keep image clear and crop unnecessary background.
```

Serial Monitor Output - Sample

```
{"presence":1,"alcohol":842,"temp":30.1}
{"status":"ALERT","buzzer":"ON","led":"RED"}
{"cloud":"ready","event_id":"demo-001"}

Use real values after running your prototype.
```

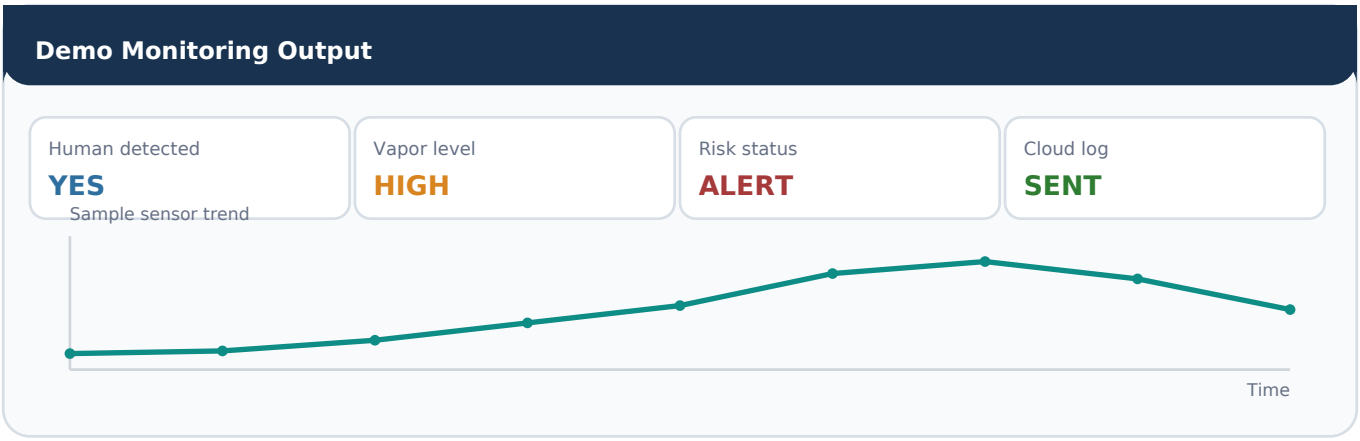

Figure 3. Sample output preview. Replace with your actual project output images when available.

Suggested images for your repository

- images/block_diagram.png - simplified architecture image.
- simulation/proteus_screenshot.png - circuit simulation screenshot.
- simulation/serial_monitor_output.png - live reading output.
- images/output_preview.png - dashboard or alert-state output.
- images/prototype_photo.png - hardware photo if available.

6. Demo Code

The following is a safe demo sketch for a prototype/simulation repository. It does not represent the complete patent implementation. Replace pin numbers, sensor names and thresholds according to your actual hardware.

```
/*
  Passive Alcohol Detection - Demo Sketch
  Public GitHub version: simplified prototype logic only.
  Replace the generic analog vapor sensor with your selected module.
*/

#define ALCOHOL_PIN 34      // Analog input on ESP32
#define PRESENCE_PIN 27     // PIR / digital presence input
#define RED_LED 14
#define GREEN_LED 12
#define BUZZER 26

const int ALCOHOL_THRESHOLD = 700;  // Calibrate using real testing
const int WARNING_THRESHOLD = 500;

void setup() {
  Serial.begin(115200);
  pinMode(PRESENCE_PIN, INPUT);
  pinMode(RED_LED, OUTPUT);
  pinMode(GREEN_LED, OUTPUT);
  pinMode(BUZZER, OUTPUT);
  digitalWrite(RED_LED, LOW);
  digitalWrite(GREEN_LED, HIGH);
  digitalWrite(BUZZER, LOW);
}

void loop() {
  int alcoholValue = analogRead(ALCOHOL_PIN);
  int presence = digitalRead(PRESENCE_PIN);

  String status = "NORMAL";

  if (presence == HIGH && alcoholValue >= ALCOHOL_THRESHOLD) {
    status = "ALERT";
    digitalWrite(RED_LED, HIGH);
    digitalWrite(GREEN_LED, LOW);
    digitalWrite(BUZZER, HIGH);
  } else if (alcoholValue >= WARNING_THRESHOLD) {
    status = "WARNING";
    digitalWrite(RED_LED, HIGH);
    digitalWrite(GREEN_LED, HIGH);
    digitalWrite(BUZZER, LOW);
  } else {
    status = "NORMAL";
    digitalWrite(RED_LED, LOW);
    digitalWrite(GREEN_LED, HIGH);
    digitalWrite(BUZZER, LOW);
  }

  Serial.print("{\"presence\":");
  Serial.print(presence);
  Serial.print(",\"alcohol\":");
  Serial.print(alcoholValue);
  Serial.print(",\"status\":\"");
  Serial.print(status);
  Serial.println("\"}");

  delay(1000);
}
```

7. Applications

Area	Use case
Campus / hostels	Non-intrusive safety monitoring in controlled areas.
Industrial workplaces	Early warning near safety-sensitive zones.
Transport checkpoints	Supportive screening at entry or monitoring points.
Event venues	Crowd safety and security support.
Research labs	Embedded sensing and IoT demonstration platform.

Expected Output States

Condition	System State	Output
Low vapor + no presence	Normal	Green LED / no buzzer
Moderate vapor reading	Warning	Warning log / optional LED indication
High vapor + presence detected	Alert	Red LED + buzzer + log event
Sensor unavailable	Fault / maintenance	Diagnostic message

These states are demonstration categories. Do not mention accuracy or final validation results in GitHub until you perform actual testing and calibration.

8. GitHub Repository Format

Use the following structure so the repository looks professional and easy to understand for recruiters, mentors and evaluators.

```
passive-alcohol-detection-system/
|-- README.md
|-- docs/
|   `-- project_summary.pdf
|-- code/
|   `-- alcohol_detection_demo.ino
|-- images/
|   |-- block_diagram.png
|   |-- workflow.png
|   `-- output_preview.png
|-- simulation/
|   |-- proteus_screenshot.png
|   `-- serial_monitor_output.png
`-- LICENSE
```

Recommended README short description

```
# Passive Alcohol Detection System

An embedded safety-monitoring prototype that detects alcohol vapor level and human presence to generate local a
```

Upload checklist

- Add README.md with project title, overview, features, technologies, working and screenshots.
- Add this PDF in docs/project_summary.pdf.
- Add demo code in code/alcohol_detection_demo.ino.
- Add clear images in images/ and simulation folders.
- Do not upload full patent draft, claims, signatures, certificates or internal comments.

9. Non-Confidential Disclosure Notice

This public project summary intentionally avoids disclosing the complete patent draft, detailed novelty, exact claim language, formal specification pages, official drawings, signatures, institutional forms and internal comments. It is suitable for a GitHub portfolio because it shows your engineering understanding without exposing sensitive IP material.

Safe to include in GitHub

- Project overview and motivation.
- General working flow.
- Simplified public-safe block diagram.
- Demo code and simulation screenshots.
- Output images after testing.
- Technology stack and applications.

Avoid uploading publicly

- Full patent draft or complete specification.
- Patent claims and exact novelty points.
- Detailed commercial implementation logic.
- Official patent drawings if your mentor/IPR cell has not approved sharing.
- Documents with signatures, registration numbers, private emails or certificates.

Final note: before making the repository public, show this PDF and the GitHub README to your mentor or IPR coordinator if the patent application is still sensitive.